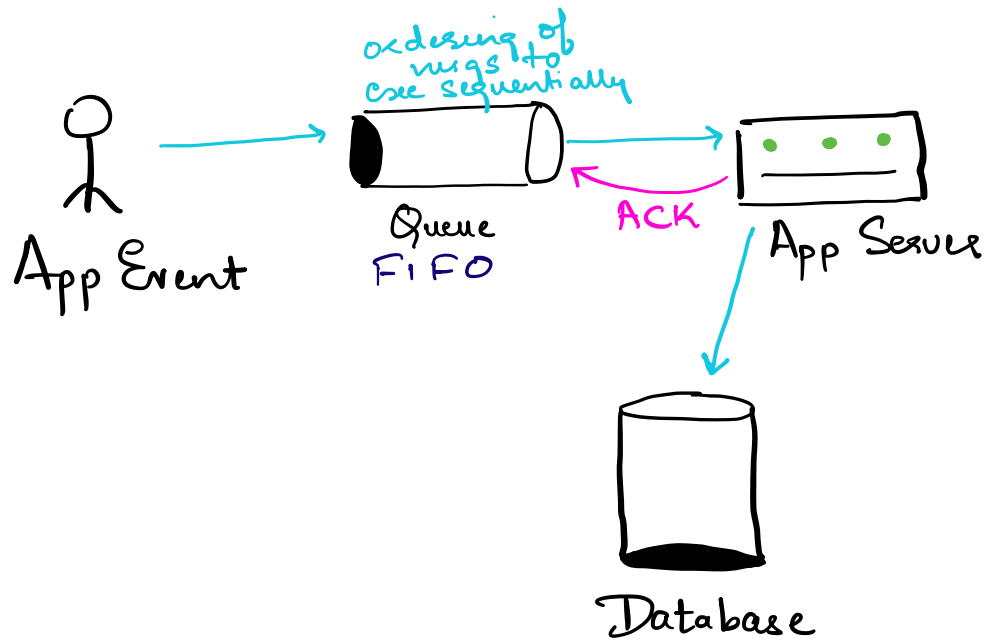
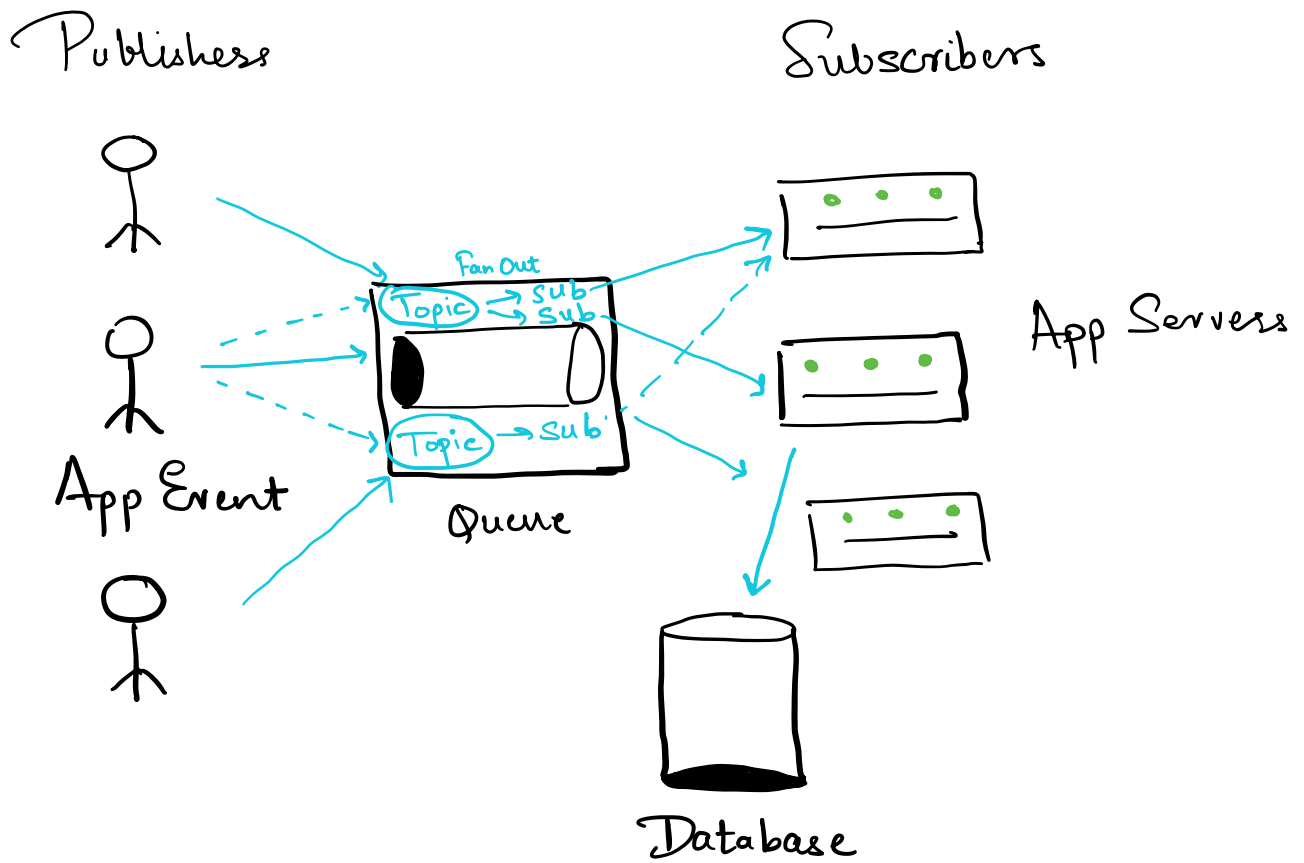


Message Queues



- X: large amount of App Events and App Server cannot handle them all together and so want to process them asynchronously we use Message Queues
eg: Payments
- X: This helps to increase the scale and queue can use FIFO or any other methods
- X: Also MQ are durable, in event of crash the data/events are persisted
- X: The way App Server gets the events is by either Pulling (keep checking if messages are in Q) or Pushing (Q keeps pushing messages to App Server on demand)
- X: Also, in order to avoid losing in messages App Server send ACK to Queue indicating the message is received so Queue can remove it from its list

Pub / Sub (Publishers / Subscribers)



Many to Many

⇒ Topics receive & store messages from publishers

⇒ Topics feed into subscription and there can be multiple subscription a single topic may feed into
Meaning, multiple application can receive from the same topic

⇒ A single topic may consume payments info and another topic may consume analytics info

⇒ We can mix & match Topics/subs the point is we do not need to change our architecture we can drop & add new API / App Server

Eg:- Rabbit MQ (Open Source)
Kafka (Open Source & Popular)
Pub/Sub (Google Cloud)